

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB 6 REPORT

Firewall Traffic Control and Forensic Verification

Course Name	CIP-B103: Network Forensics Fundamentals
Lab Title	CIP-B103 Lab 6: Firewall Traffic Control and Forensic Verification
Student Name	Wihbedimbom Sandrin
Student ID	C11/26/DFIT/17292
Instructor Name	Dr. Sarah James

2. Executive Summary

This lab evaluated the operational mechanics and forensic footprint of host-based firewall rules using Linux iptables. An Apache web server was deployed to serve

an HTTP endpoint. A baseline capture confirmed normal HTTP connections over port 80. A target-specific iptables rule (-j DROP) was subsequently applied to block traffic originating from a designated client IP. Network captures confirmed that dropped packets result in TCP SYN retransmissions and client timeouts without triggering server-side TCP responses. Correlation between iptables byte counters and packet capture timelines confirmed rule activation and hit verification.

3. Lab Objectives

- Differentiate between host-based and network-based firewall deployment architectures.
- Analyze iptables structures, including INPUT, OUTPUT, and FORWARD chains.
- Configure, verify, and remove targeted packet-filtering rules in iptables.
- Analyze packet captures to distinguish between DROP and REJECT firewall actions.
- Correlate client application behaviors (curl timeouts) with firewall rule counters and PCAP evidence.
- Execute safe firewall administration and complete ruleset restoration procedures.

4. Tools and Resources Used

- **Apache2 Web Server:** Linux daemon hosted on target server providing HTTP resources on port 80.
- **iptables / iptables-save:** Linux kernel Netfilter packet management utilities.
- **UFW (Uncomplicated Firewall):** Disabled prior to testing to avoid translation conflicts with raw iptables.
- **curl:** Command-line HTTP client used to generate web requests and record connection timing.
- **tshark / Wireshark:** Network capture and traffic inspection tools.

5. Methodology

5.1 Environment Setup and Baseline Backup

1. Disabled UFW to prevent rule inheritance issues and backed up current iptables state:

```
(root@icdfa)-[~/CIP-B103-Lab5]
# # 1. Create workspace directory structure
mkdir -p ~/CIP-B103-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab6

# 2. Disable UFW to prevent conflicts and back up initial iptables state
sudo ufw disable
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt

# 3. Setup Apache test web page and check port 80 listener
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
sudo: ufw: command not found
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source                   destination
e3b0c44298fc1c149afb4c8b996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1></body></html>
LISTEN 0      511          *:80          *:.*          users:((("apache2",pid=3856,fd=4),("apache2",pid=3855,fd=4),("apache2",pid=3854,fd=4),("apache2",pid=3853,fd=4),("apache2",pid=3852,fd=4),("apache2",pid=1034,fd=4)))
```

2. Provisioned Apache web page and validated port 80 listener:

```
(root@icdfa)-[~/CIP-B103-Lab6]
# sudo iptables -F INPUT
sudo systemctl restart apache2

# # Verify apache page
curl -v http://127.0.0.1/firewall_lab.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 44714
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 07 Aug 2026 19:49:35 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 07 Aug 2026 19:30:41 GMT
< ETag: "58-6587a0a403cc7"
< Accept-Ranges: bytes
< Content-Length: 88
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1></body></html>
* Connection #0 to host 127.0.0.1:80 left intact
```

5.2 Part B: Allowed Baseline Capture

Initiated tshark capture on the server and executed client HTTP GET request using curl:

```
(root@icdfa) [~/CIP-B103-Lab6]
# SERVER_IP="192.168.64.129"
sudo tshark -i eth0 -f "tcp port 80" -a duration:10 -w evidence/http_allowed.pcapng &
curl -v --connect-timeout 5 "http://$SERVER_IP/firewall_lab.html"
[1] 6239
* Trying 192.168.64.129:80 ...
* Established connection to 192.168.64.129 (192.168.64.129 port 80) from 192.168.64.129 port 33970
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.64.129
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 07 Aug 2026 20:05:28 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 07 Aug 2026 19:30:41 GMT
< ETag: "58-6587a0a403cc7"
< Accept-Ranges: bytes
< Content-Length: 88
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1></body></html>
* Connection #0 to host 192.168.64.129:80 left intact
```

5.3 Part C & D: Applying DROP Rule and Capturing Blocked Traffic

1. Inserted a target rule at position 1 in the INPUT chain to silently drop TCP port 80 traffic from client 136.160.215.15:

```
(root@icdfa) [~/CIP-B103-Lab6]
# sudo iptables -I INPUT 1 -s 136.160.215.15 -p tcp --dport 80 -j DROP
sudo iptables -I INPUT 2 -s 192.168.64.129 -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 67 packets, 6186 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *       136.160.215.15       0.0.0.0/0           tcp dpt:80
2      0      0 DROP      tcp  --  *      *       192.168.64.129       0.0.0.0/0           tcp dpt:80
```

2. Captured traffic during blocked client attempt and checked iptables hit counters:

```
(root@icdfa) [~/CIP-B103-Lab6]
# sudo tshark -i eth0 -f 'host 136.160.215.15 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
curl -v --connect-timeout 10 http://136.160.215.194/firewall_lab.html
wait
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
[1] 6291
* Trying 136.160.215.194:80 ...
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
* Connection timed out after 10002 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10002 milliseconds

[1] + done      sudo tshark -i eth0 -f 'host 136.160.215.15 and tcp port 80' -a duration:35 -
Chain INPUT (policy ACCEPT 70 packets, 6326 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *       136.160.215.15       0.0.0.0/0           tcp dpt:80
2      0      0 DROP      tcp  --  *      *       192.168.64.129       0.0.0.0/0           tcp dpt:80
```

5.4 Part E & F: Rule Deletion & Network Restoration

Removed specific rule by rule specification and verified access restoration:

```

(root@icdfa)-[~/CIP-B103-Lab6]
# sudo iptables -D INPUT -s 136.160.215.15 -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
curl -v --connect-timeout 5 http://136.160.215.194/firewall_lab.html
Chain INPUT (policy ACCEPT 70 packets, 6326 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *      192.168.64.129       0.0.0.0/0           tcp dpt:80
* Trying 136.160.215.194:80 ...
* Connection timed out after 5002 milliseconds
* closing connection #0
curl: (28) Connection timed out after 5002 milliseconds

```

6. Screenshots and Evidence

- Figure 1 (Methodology 5.1):** Terminal view showing workspace creation, ufw: command not found, iptables-save SHA-256 hash (e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855), HTML payload creation, and Apache2 listener on *:80 (PID 3856, 3855, 3854, 3853, 3852, 1034).
- Figure 2 (Methodology 5.2):** Execution of curl -v --connect-timeout 5 http://192.168.64.2/firewall_lab.html returning Connection Refused after 71 ms prior to active listener binding.
- Figure 3 (Methodology 5.3):** Terminal view showing insertion of DROP rule 1 for 136.160.215.15 and DROP rule 2 for 192.168.64.129 into the INPUT chain.
- Figure 4 (Methodology 5.3):** Execution of tshark background capture and curl -v --connect-timeout 10 http://136.160.215.194/firewall_lab.html returning curl: (28) Connection timed out after 10002 milliseconds.
- Figure 5 (Methodology 5.4):** Deletion of DROP rule for 136.160.215.15 leaving Rule 1 for 192.168.64.129, followed by curl timing out after 5003 milliseconds.

7. Analysis and Findings

7.1 PCAP Evidence Comparison: Allowed vs. Blocked

Forensic Indicator	Unreachable / Allowed Baseline (Screenshot 5.2)	Blocked Capture (Screenshot 5.3)
Client Initial SYN	Transmitted from 192.168.64.129:43876 to 192.168.64.2:80	Transmitted from client port 37204 to target port 80

Server Response	TCP RST-ACK returned (Connection refused after 71 ms)	Absent (Packet silently dropped at Netfilter INPUT chain)
3-Way Handshake	Failed immediately via socket RST	Failed via transport layer SYN timeout
HTTP Payload Transfer	Absent (Connection refused)	Absent (Transport connection never established)
Client Application Result	curl: (7) Failed to connect to 192.168.64.2 port 80 after 71 ms	curl: (28) Connection timed out after 10002 milliseconds
Netfilter Chain Policy State	Chain INPUT (policy ACCEPT 0 packets, 0 bytes)	Chain INPUT (policy ACCEPT 6 packets, 280 bytes)

7.2 Forensic Interpretation & Questions

- **Why DROP Causes Retransmissions & Timeout:** The DROP target silently discards matching packets at the kernel level without generating TCP RST or ICMP error frames. The client stack assumes packet loss and retransmits SYN packets until the connection timeout threshold (10002 ms) is reached.
- **DROP vs. REJECT Differences:** A REJECT target actively returns a TCP RST packet or ICMP Port Unreachable message. In terminal captures, REJECT appears as an immediate Connection Refused (71 ms), whereas DROP causes the application to hang until timing out.
- **Corroborative Value of Firewall Counters:** iptables packet and byte counters provide system-side evidence verifying that packets matched firewall filter

criteria, confirming rule enforcement even if application-level daemon logging is disabled.

- **Service Down vs. Firewall Blocked:** When Apache is down or not bound to an IP socket, the OS kernel returns an immediate TCP RST-ACK (Connection refused after 71 ms). When a DROP firewall rule is active, zero response packets are generated, resulting in a 10002 ms client timeout.

8. Challenges and Solutions

- **Challenge:** Deleting the DROP rule for 136.160.215.15 left Rule 1 for 192.168.64.129 active, causing subsequent curl requests to continue timing out after 5003 ms.
- **Solution:** Execute `sudo iptables -F INPUT` to completely flush custom filtering rules, returning the host Netfilter stack to a clean default ACCEPT state.

9. Conclusion

This lab demonstrated how host-based firewalls inspect incoming packets and alter network signatures. Analyzing the distinction between TCP RST frames (Connection refused after 71 ms) and kernel DROP behavior (Connection timed out after 10002 ms) provides definitive forensic evidence during network traffic investigations.

10. Recommendations

- Flush custom filtering chains (`sudo iptables -F`) prior to testing to prevent residual rule conflicts.
- Position blocking rules explicitly at the top of chains (`-I INPUT 1`) to ensure priority evaluation.
- Export baseline rulesets (`iptables-save`) prior to executing administrative firewall changes.

11. References

- Netfilter Project. iptables Documentation. Retrieved from <https://netfilter.org/>
- Linux Man Pages: iptables(8), curl(1).